ASSIGNMENT 5 DevOps and CI/CD

The Role of DevOps and CI/CD in Automating the University Room Booking Research Project

Sultan Turan | CSE-2502M
Applied Software Development Project
Instructor Nurzhamal Kashkimbayeva
Astana IT University | 22 October 2026

Part 1 Analytical report

1 Project context and the purpose of DevOps

The university room-booking project needs dependable software and a repeatable way to evaluate its pilot. Assignment 4 proposed a service for 40 rooms, with a four-week pilot following development. These remain planning assumptions, not observations from the university [1]. In this assignment, I focus on a small Python tool that summarises booking durations. It provides a manageable example of research automation without implementing the entire booking system.

DevOps combines development and operations through shared responsibility, automation and feedback. Lecture 7 presents these as working practices rather than simply a collection of tools [2]. For this project, developers, the tester and the IT administrator should agree how a change is checked, released and investigated when it fails. Facilities staff also need to clarify what a booking record means, because technically correct calculations can still answer the wrong question.

The main benefit is reducing variation in routine work. A researcher should not need to remember a different sequence of commands for each analysis. Versioned scripts and automated checks make that sequence visible to the team. Small, frequent changes also make faults easier to trace than a large release assembled just before the pilot.

However, automation has a cost. Somebody must maintain the pipeline, review failures and update the environment. For a small university team, the useful starting point is one repository, one container image and a short pipeline. A cluster or a large monitoring platform would need a separate justification. DevOps succeeds here when the team can confidently repeat and explain its work, not when it installs the greatest number of tools.

Part 1 Analytical report continued

2 Continuous integration and continuous delivery

Continuous integration means regularly combining changes in a shared repository and checking them automatically. In this example, a pushed change starts unit tests before the application image is built. The tests check known answers and invalid input, including missing values and nonfinite numbers. A failed check must stop later stages; otherwise, a green build would give misleading confidence about the analysis.

Continuous delivery keeps a tested version ready for release, while continuous deployment also performs the release automatically. I use automatic deployment only to a local test service. Publishing the real booking system to university users would still require an agreed release decision, permission checks and recovery procedures. Lecture 7 makes the same distinction between a releasable version and an automatic production release [2].

The practical pipeline is stored in a Jenkinsfile alongside the Python code. Its stages test commits, build an image, run the sample analysis and deploy the result to the local test service. This makes the procedure reviewable through Git. Jenkins supports both scheduled SCM checks and event-based triggers [3]. Local polling is adequate for this exercise because it avoids exposing a laptop to incoming webhooks. It introduces a short delay, and the server must remain running.

3 Reproducibility and reliability in the pilot

For a repeatable result, code alone is insufficient. The team also needs the exact input bytes, the selected column, its units and the software environment. The tool records an input SHA-256 hash, while Jenkins retains the Git revision, output JSON and Docker image ID. The container base is pinned by digest so that a moving image tag does not silently change the Python environment [4].

These records support checking whether two runs used the same materials. They do not prove that the data represents actual room use. Five synthetic bookings lasting 30, 60, 90, 60 and 120 minutes have a mean of 72 minutes. That calculation cannot establish utilisation: utilisation also needs the observation period, room availability and a rule for cancellations or overlapping records. A reservation may not mean that anybody occupied the room.

Reliability therefore has two levels. Software tests can detect a broken calculation or rejected input. Research review must assess the sampling and interpretation. Keeping these responsibilities distinct prevents a successful pipeline from being presented as evidence that the university pilot achieved its goals.

Part 1 Analytical report continued

4 Docker Jenkins and Kubernetes in research

Docker packages an application and its runtime in an image. For the booking study, a collaborator can execute the same CSV summariser without manually installing Python packages. The application uses only the standard library and runs as a non-root user. A scientific example would be sharing the environment for a simulation post-processing script, together with a small reference dataset and expected output. Digest pinning improves environment traceability, but hardware, architecture and external data must still be recorded [4].

Jenkins coordinates the sequence and preserves evidence of execution. In an engineering project, it could run regression tests when a numerical solver changes, then compare a small benchmark against a known result. For the booking tool, it checks input handling and descriptive statistics before producing the deployable image. The useful output is both the tested software and an inspectable record of which revision produced the analysis. A failed stage gives the team a concrete starting point for investigation [3].

Kubernetes addresses a different problem: managing container workloads across a cluster. A Kubernetes Job is suitable for a finite batch calculation and can retry failed Pods [5]. For example, a larger room-planning study could run independent simulations for several timetable scenarios as Jobs, with explicit resource limits and durable output storage. A long-running booking API would instead suit a Deployment and Service. Retried calculations must avoid duplicating or overwriting completed outputs.

I would not introduce Kubernetes for this five-row analysis. One Docker host is enough, and the operational effort would exceed the benefit. A cluster becomes reasonable if the research team has many independent jobs, substantial compute demand or an existing managed environment. It also does not remove the need to manage data access, backups and application failures. Container restarts alone cannot guarantee service availability or correct results.

The three tools are therefore complementary. Docker defines the execution environment, Jenkins controls when checks and delivery steps happen, and Kubernetes can manage execution at larger scale. The practical work deliberately implements the first two and a small local deployment. Kubernetes is evaluated as a possible future tool rather than claimed as part of the running system.

For real pilot records, the team should keep identifying data out of a public repository and restrict Jenkins access. The demonstration contains invented booking and room identifiers only. A shared CI installation would also need isolated build agents, since access to a Docker daemon gives pipeline code extensive control over its host.

Part 1 Conclusion and references

5 What DevOps solves for the scientific team

For this project, DevOps reduces forgotten test steps, inconsistent environments and uncertainty about which code generated a result. It also makes handover easier: the IT administrator can inspect the same deployment procedure that the developer used. When an analysis changes unexpectedly, the team can compare the input hash, commit and image instead of relying on memory.

The approach does not replace the wider acceptance criteria from Assignment 4. Tests of a CSV tool cannot demonstrate conflict-free reservations, satisfactory response times, successful backup restoration or reduced staff effort [1]. Those questions require separate system tests and pilot evidence. The analysis also needs documented rules for invalid durations, cancellations and actual attendance before it can support operational decisions.

My conclusion is that a modest CI/CD pipeline is appropriate for this stage. It gives the team repeatable checks and a traceable analysis result, while leaving a clear boundary between a classroom demonstration and production use. The next useful improvement would be testing a reviewed anonymised pilot export against agreed data rules. Adding orchestration would come later, if the workload justifies it.

References

[1] Turan, S. Assignment 4 Budget Planning and Justification: University Room Booking System. 15 October 2026, pp. 1-2 and 6. Supplied course assignment.

[2] Lecture 7 DevOps and CI/CD. Supplied course handout, pp. 1-3, 6-11. Principles, pipeline stages and research reproducibility.

[3] Jenkins. Pipeline Syntax. Official documentation, especially stages, triggers and post actions. jenkins.io/doc/book/pipeline/syntax/

[4] Docker. Building best practices. Official documentation, especially pinning base images and USER. docs.docker.com/build/building/best-practices/

[5] Kubernetes. Jobs. Official documentation on finite workloads and retries. kubernetes.io/docs/concepts/workloads/controllers/job/

Online documentation accessed 22 October 2026. The practical code continues the author's Assignment 3 CSV summary repository; its URL and attribution are included in the project README.

Part 2 Practical report

1 Implementation and steps

I continued the Python CSV summariser from Assignment 3 and added a synthetic booking-duration dataset. The project is stored publicly at:
github.com/Eager-coder/booking-devops-assignment-5
The repository includes the Dockerfile, Jenkinsfile, tests, Compose configuration and Jenkins setup script.

First, I retained the seven existing test methods and added two checks for booking statistics and the exact input hash. Second, I containerised the tool using a digest-pinned Python 3.12 slim image. The runtime has no third-party Python dependencies and uses a non-root account. Third, I started Jenkins in Docker on localhost port 8085, with a login and a persistent volume. Port 8080 was already occupied, so I used a separate port.

The Jenkins job reads the Jenkinsfile from the main branch. After an initial manual build, SCM polling checks GitHub every minute. Each new pushed commit is tested; if polling groups several commits, the test script checks each one since the previous successful build. Only after passing tests does Jenkins build the latest image, tagged with its full Git SHA. Unpushed local changes are outside this trigger.

The pipeline then executes the image and archives the JSON output, commit and image ID. Finally, it deploys the same image as a local HTTP test service on port 8086. An HTTP smoke test checks that the published JSON contains five bookings and a mean of 72 minutes. This provides the optional automatic CD demonstration without a registry or Kubernetes cluster.

2 Difficulties and observed results

The initial Jenkins bootstrap used an unsupported Git plugin branch-setting method, leaving the job without a pipeline definition. I corrected it to update the existing branch specification and restarted Jenkins. The failed setup builds remain visible in the build history. I also used Docker copy operations for test sources and results because Jenkins runs inside a container while the Docker daemon runs on the host; their workspace paths are not interchangeable.

Build 3 passed after the setup fix. Build 4 then started automatically from SCM change 6a98207 and passed all four stages, with nine tests and the HTTP check successful. The output had n = 5, mean = 72, median = 60 and sample SD = 34.2053 minutes. The run used Jenkins 2.580.1, Docker 29.8.2 on arm64 and Python 3.12.15.

3 Conclusion and limits

The workflow now connects Git changes to tests, a Docker image and a running test deployment. It improves repeatability without changing the scope of the booking project. The local server has a brief replacement outage and no automatic rollback. Jenkins uses the host Docker socket for this trusted single-user lab; a shared deployment would need isolated agents. These tests validate the analysis tool, not the full booking service.

Part 2 Screenshots and execution evidence

Figure 1 Working Jenkins pipeline

Build 4 (commit 6a98207) completed successfully after an automatic SCM trigger on 22 October 2026. Build 3 was the successful initial run. Earlier setup failures remain in the Jenkins build history.

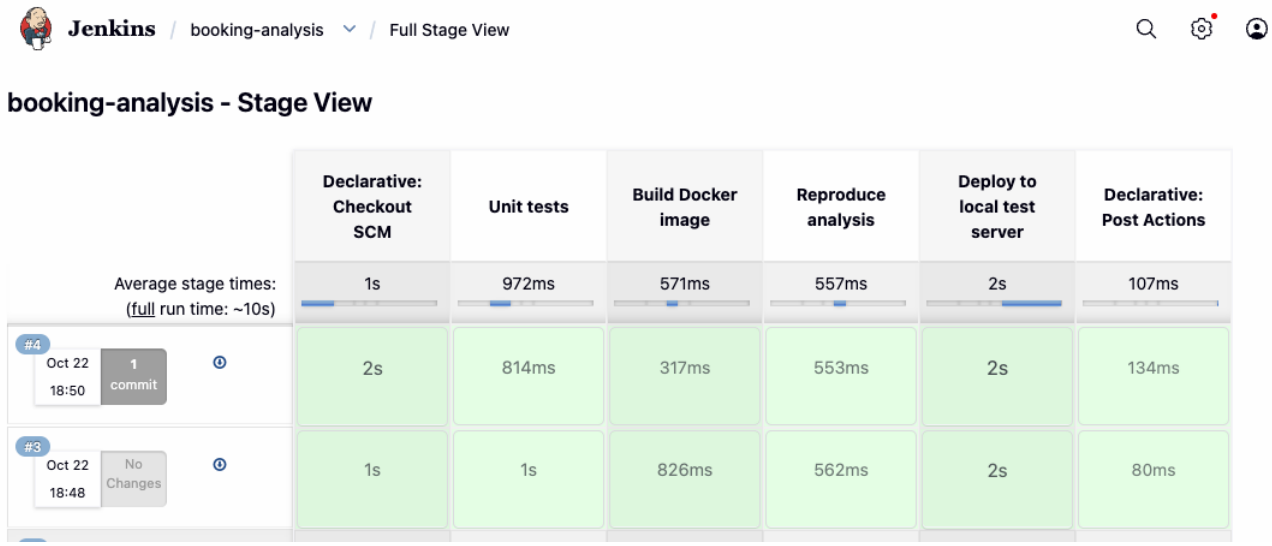
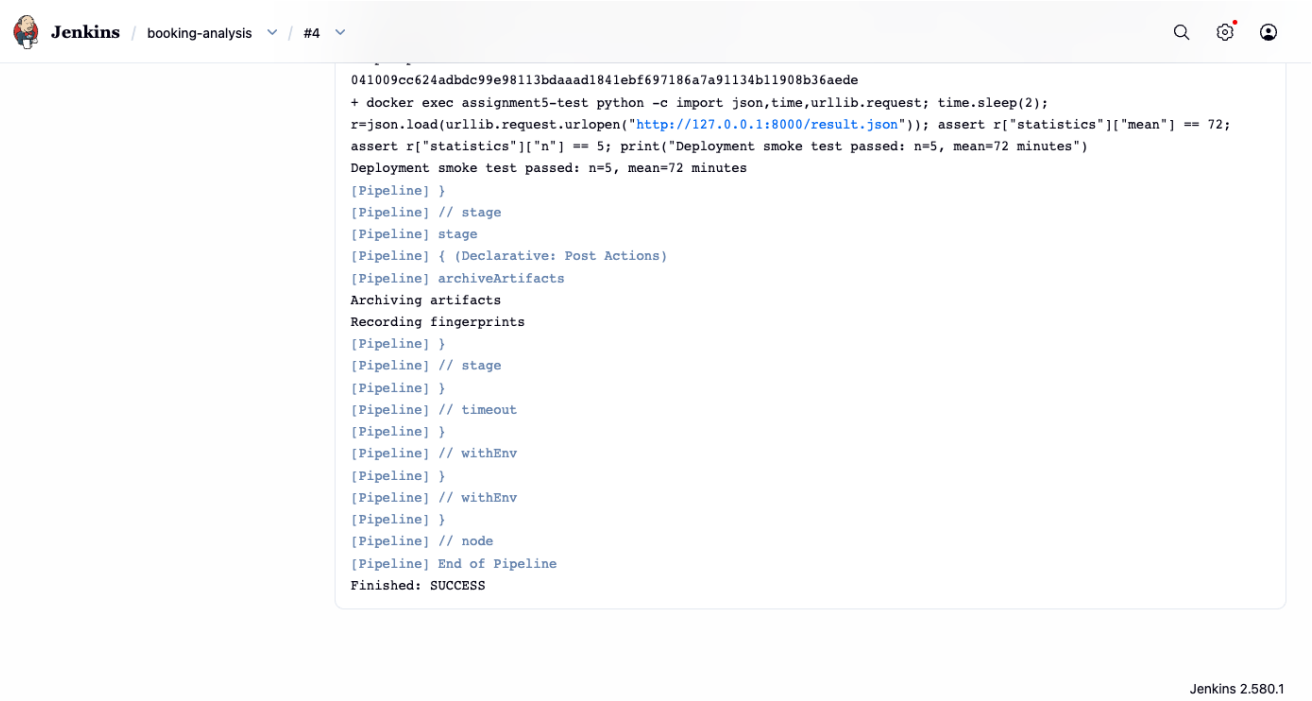


Figure 2 Successful deployment and final build result

The console records the HTTP smoke test and final SUCCESS result. The separate build-details screenshot and metadata in evidence/ identify the automatic SCM trigger. Full-resolution screenshots and logs are available in the repository.



Setup commands, local access instructions and cleanup steps are in the repository README.